Pattern Recognition

Albi Sadikaj
albi.sadikaj@edu.unifi.it

Cosimo Sgambelluri
cosimo.sgambelluri@edu.unifi.it

Abstract

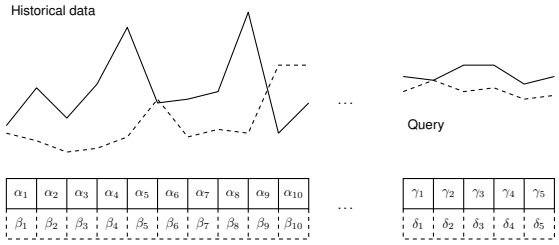
Pattern recognition within large datasets is a computationally intensive task, requiring the continuous evaluation of query sequences against target data to identify optimal matches. This similarity is typically quantified using distance metrics such as the Sum of Absolute Differences (SAD). This paper investigates the acceleration of a sliding-window SAD algorithm by leveraging both multi-core CPU (OpenMP) and massively parallel GPU (CUDA) architectures. We present a comprehensive performance evaluation comparing these parallel implementations against a sequential C++ baseline.

1. Introduction

The primary objective of pattern recognition in time series analysis is to locate a specific query sequence within a much larger dataset. Since perfect data alignment is statistically improbable, this task relies on proximity metrics, such as the Sum of Absolute Differences (SAD), to quantify similarity and isolate the closest match.

However, applying a sliding-window SAD calculation to large datasets exposes severe limitations in sequential computing. The overlapping nature of the data causes high memory bandwidth saturation, making sequential C++ implementations impractically slow for large-scale queries. To solve this, our study implements and analyzes two parallelized alternatives: an OpenMP approach utilizing CPU multi-threading, and a CUDA approach harnessing GPU grid architecture.

This report benchmarks these systems by evaluating how thread count, query length, and concurrent queries impact performance. We compare parallel implementations against a sequential baseline to quantify speedup and identify hardware limitations.



1.1. Sum of Absolute Differences

The core metric utilized to determine the similarity between a candidate window in the time series and a specific query is the Sum of Absolute Differences (SAD). SAD is a widely adopted similarity measure in signal processing and pattern recognition due to its mathematical simplicity.

Given a single-channel time series T of length N and a query sequence Q of length M (where $M \leq N$), the SAD for a sliding window starting at index i is mathematically defined as:

$$SAD(i) = \sum_{j=0}^{M-1} |T_{i+j} - Q_j|$$

where $0 \leq i \leq N - M$. A lower SAD value indicates a higher degree of similarity between the query and the current window of the time series, with a value of exactly zero representing a perfect, identical match.

1.2. Computational Challenges

The pattern recognition task over multidimensional time series is computationally demanding. The naive sliding window approach has a time complexity of $O((N - M + 1) \times M \times Q \times C)$, which simplifies to $O(N \times M \times Q \times C)$ for $N \gg M$. Here, N is the length of the time series, M is the query length, Q is the number of queries, and C is the number of channels. Given our dataset of 2,000,000 points and four queries of length 819, this naive evaluation translates into nearly 33 billion floating-point operations, highlighting the necessity for parallelization.

Furthermore, the task presents two primary architectural challenges:

- **Memory bandwidth bottlenecks:** because sliding windows continuously evaluate overlapping data points, a naive implementation will repeatedly fetch the same values from memory. If not carefully managed, this pattern quickly thrashes CPU caches and saturates GPU memory bandwidth.

- **Thread divergence & load imbalance:** early abandonment optimizations drastically reduce compute cycles by halting unpromising SAD calculations early. However, this creates highly variable processing times per window, resulting in severe load imbalances across CPU threads and thread divergence within GPU warps.

2. Implementations

2.1. Sequential

The baseline sequential implementation in C++ focuses on establishing a robust and correct algorithm. Rather than an Array of Structures (AoS), we used a Structure of Arrays (SoA) layout (`TimeSeriesSoA`), storing each of the 5 channels in contiguous memory arrays. This layout promotes data locality and facilitates auto-vectorization by the compiler.

The baseline algorithm employs a standard sliding window approach, sequentially iterating through the dataset to compute the complete Sum of Absolute Differences (SAD) for each window. If the calculated SAD is lower than the global minimum found thus far, the minimum is updated. While incorporating an early abandonment optimization—terminating the calculation as soon as the running sum exceeds the current minimum—seemed appealing, it severely degraded performance in practice. Evaluating this conditional break after every element introduces a high frequency of branch instructions into the innermost loop. The overhead of these continuous branch evaluations outweighs the computational cycles saved by stopping early.

2.2. Parallel OpenMP

The OpenMP implementation introduces thread-level parallelism targeting multi-core CPUs. For each time series channel, the sliding window search space is distributed across available threads using separate `#pragma omp parallel` and `#pragma omp for` directives. Each thread maintains thread-local trackers for the best SAD and its index, which are merged into a global result via a `critical` section after the parallel region completes. The five channels are processed sequentially in the outer loop, and — since the function is called once per query — query-level parallelism is not exploited.

To maximize hardware utilization and maintain correct synchronization, the implementation relies on three strategies:

1. Dynamic scheduling: to address the load imbalance caused by the early abandonment optimization, we utilized dynamic scheduling strategies with a tuned chunk size (`schedule(dynamic, 256)`). Because early abandonment causes different windows to require vastly different computation times, static scheduling would leave threads idle. Dynamic scheduling ensures that threads finishing their assigned chunks early—due to rapid SAD threshold breaks—are continuously fed new blocks of work, minimizing idle CPU time and significantly improving overall parallel efficiency.

2. Chunked SIMD vectorization: a major challenge is that early abandonment (which requires frequent conditional branching) inherently conflicts with CPU vectorization (SIMD), which requires uninterrupted loop execution. To solve this, we used a chunking mechanism. The SAD calculation for a query is divided into sub-blocks of 64 elements. Within each 64-element block, we force vectorization using `#pragma omp simd reduction(+:chunk_sad)`, allowing the CPU to process multiple floating-point absolute differences simultaneously. The early abandonment check (`sad >= local_best_sad`) is then evaluated only at the boundary of these chunks. This effectively balances the high throughput of SIMD instructions with the algorithmic savings of early termination.

3. Thread-local state: to prevent race conditions without incurring the massive overhead of locking every iteration, we implemented thread-local state tracking. Each thread maintains its own `local_best_sad` and `local_best_idx`. As a thread processes its assigned windows, it only updates these private variables. Once the parallel region completes, a single `#pragma omp critical` section is used to safely merge the thread-local optimums into the global best result for that channel.

2.3. Parallel CUDA

The CUDA implementation shifts the computation to a massively parallel GPU architecture, fundamentally changing the algorithmic approach. Most notably, the early abandonment optimization utilized in the CPU implementation is, from a point of view of the architecture, incompatible with the GPU's SIMT execution model. In CUDA, threads are scheduled in groups of 32, known as *warps*. Every thread within a warp is physically bound to execute the exact same instruction at the same time. If an early break condition causes threads to diverge—where some exit the loop early while others continue calculating the SAD—the hardware must serialize the execution of the divergent paths. To avoid the severe performance penalty of *warp divergence*, the CUDA kernel calculates the full SAD for every window uninterrupted.

To offset the algorithmic cost of calculating full windows

and to mitigate the severe global memory bandwidth bottlenecks inherent to the task, the implementation explicitly manages the GPU's memory hierarchy and execution topology:

- **2D grid scheduling:** instead of looping over queries sequentially on the host, the problem space is mapped onto a 2-dimensional execution grid. The X-axis distributes the starting indices of the dataset (`((total_elements + BLOCK_SIZE - 1) * BLOCK_SIZE`), while the Y-axis distributes the evaluation of different queries (`NUM_QUERIES`). Configured with `BLOCK_SIZE = 512` threads, this mapping ensures maximum multiprocessor occupancy and allows the GPU to evaluate multiple queries simultaneously.
- **Constant memory for queries:** because queries are strictly read-only and accessed uniformly by all threads in a warp, they are explicitly copied to the GPU's `__constant__` memory space (e.g., `c_q1` through `c_q5`). This leverages the GPU's hardware broadcast mechanism, allowing a single memory fetch to instantly serve all 32 threads in a warp without saturating standard memory bandwidth.
- **Shared memory:** the sliding window approach requires reading the same time series data points overlappingly. To prevent redundant reads from slow global memory (`d_c1`, etc.), we utilize dynamically allocated `__shared__` memory (`extern __shared__ float s_data[]`). Threads within a block cooperatively fetch a continuous chunk of data (`BLOCK_SIZE + QUERY_LENGTH`). After a `__syncthreads()` barrier, all threads perform their SAD calculations.
- **Instruction optimization:** within the kernel, the innermost SAD calculation loop over `QUERY_LENGTH` utilizes the `#pragma unroll 8` directive. This instructs the `nvcc` compiler to explicitly expand the loop into discrete arithmetic instructions, effectively reducing loop-control branch overhead.

3. Experimental Setup

3.1. Hardware Specifications

The performance evaluations were all conducted on the same workstation. The hardware specifications can be found at Table 1.

Component	Specification
CPU	AMD Ryzen 7 7840HS
Cores	8
Threads	16
Cache	512 KB L1 (Data/Instr.), 8 MB L2, 16 MB L3
RAM	16 GB DDR5 5600 MHz
GPU	NVIDIA RTX 4060 Mobile 8 GB

Table 1: Hardware configuration for benchmarking.

3.2. Software Environment

The project was built within a Windows 11 environment utilizing `CMake` 4.0.2 as the primary build system. The CPU sequential and OpenMP implementations were compiled using GCC 15.2.0. To maximize CPU performance, we enabled the following flags:

- `-O3`: instructs the compiler to enable all the optimization flags to try and increase performance without altering the results.
- `-march=native`: instructs the compiler to try and use AVX2 or even AVX512, if supported.
- `-ffast-math`: instructs the compiler to break certain rules to make calculations faster. Some rules are: assumption of finite math, IEEE compliance in the order of instructions and no signed zero.
- `-fopenmp`: enables the use of OpenMP directives to support multi-threading.

The CUDA implementation was compiled via the NVIDIA CUDA Compiler (NVCC) toolkit version 13.0.88, specifically targeting compute capability 8.9. The flags enabled for the NVCC compiler are similar to the previous ones. The only flags that weren't mentioned already are:

- `-Xcompiler /O2`: tells the NVCC compiler that, when it has to ask the MSVC compiler to compile something, to use the `/O2` flag.
- `--ptxas-options=-v`: asks the compiler to print in the console memory usage statistics like: registry count, constant and shared memory size is being used.

As for the pure C++ file in the CUDA implementation compiled using the MSVC compiler, the flags used are: `/O2` (equivalent of `-O3`), `/fp:fast` (equivalent of `-ffast-math`) and `/arch:AVX2` that enables the use of AVX2 instructions.

3.3. Dataset and Queries

A synthetic dataset simulating realistic multidimensional data was generated using the `generate_data.py` script. The time series consists of $N = 2,000,000$ continuous data points recorded across $C = 5$ distinct channels.

From this baseline data, sets of queries with varying predefined lengths were extracted. To ensure reproducible evaluation and diverse coverage across the dataset, the starting indices for these extractions were deterministically and evenly spaced throughout the time series. The extracted segments across all five channels were then perturbed with uniform random noise, generated via a fixed-seed "custom" pseudo-random number generator, to simulate inaccuracies and test the robustness of the SAD distance metric.

3.4. OMP Correctness Validation

To ensure the mathematical correctness of the parallel implementations, the program incorporates an automatic correctness validation step during every configuration run. The outputs of the parallel algorithm are compared against the results from the baseline sequential algorithm.

This validation is performed by the `SameResults` function and requires two conditions to be met for every query:

- The parallel algorithm must identify the exact same `best_indices` as the sequential baseline for the minimum SAD across all time series channels.
- Because multi-threaded parallel reductions can change the order of operations and introduce minor floating-point precision differences, an exact equality check is not viable. Instead, the absolute difference between the sequential and parallel SAD values must fall within a defined tolerance threshold (atol $= 10^{-2}$).

If either condition fails, the execution is immediately halted with an error, ensuring that speedups are only recorded for strictly accurate computations.

3.5. CUDA Consistency Validation

Unlike OMP, the CUDA implementation guarantees correctness by verifying results consistency across multiple execution runs.

Specifically, it maintains an array of the minimum SAD indices (`last_best_indices`) from the previous kernel execution. At the end of every new run, the newly computed `best_indices` for all queries and channels are compared against the prior run's results.

If any index differs between iterations, the program flags an error.

3.6. Tested Parameters

To evaluate the scalability and efficiency of our parallel approaches, we varied the following execution parameters:

- **Thread count:** evaluated from 1 up to 16, representing double the number of physical CPU cores.
- **Query count:** scaled from 1 to 4 queries.
- **Query length:** tested at 128, 256, 512, and 819. The maximum length of 819 is a hard architectural constraint; any larger value would cause the total query data to exceed the GPU's 64 KB `__constant__` memory capacity.

4. Results Analysis

4.1. Sequential Baseline

The sequential C++ implementation serves as the primary reference point for all speedup calculations. As demonstrated in Table 2, execution time exhibits near-perfect linear scaling with query length. Similarly, because additional queries are processed sequentially one after the other, increasing the query count acts as a direct multiplier on the overall execution time.

	Query Count			
Query Length	**1**	**2**	**3**	**4**
128	77.30	157.94	236.34	329.39
256	149.33	307.10	458.30	611.55
512	288.83	593.31	889.30	1239.03
819	487.68	962.04	1486.04	1971.96

Table 2: Average sequential execution times (ms) across varying query lengths and query counts.

4.2. OpenMP Version

4.2.1 Single-thread overhead

A predictable performance penalty is immediately apparent at $T = 1$ thread: the OpenMP implementation is roughly $3\times$ *slower* than the sequential baseline across all configurations. For example, at query length 819 and query count 1, the sequential wall time is 487.7 ms whereas OpenMP with one thread requires 1416.3 ms as it can be seen in Table 3. This overhead stems from the cost of initializing the OpenMP thread pool every time we move to another data channel, setting up the dynamic scheduler, and maintaining thread-local state — none of which yield any parallelism benefit at $T = 1$.

An alternative parallel formulation was explored in which the thread pool is forked once for the entire workload rather than once per time series channel. While this reduced overhead significantly at $T = 1$ (approximately $3\times$ improvement), it introduced explicit inter-series barriers that caused consistent regressions at $T = 4$ and $T = 8$ — the most relevant thread counts on the target hardware.

Query Size	Sequential	OMP $T = 1$
128	77.30	291.16
256	149.33	493.91
512	288.83	896.58
819	487.68	1416.31

Table 3: Sequential and OpenMP ($T = 1$) average wall times (ms).

Query length	T=1	T=2	T=4	T=8	T=16
128	0.27×	0.97×	2.05×	2.85×	4.59×
256	0.30×	1.07×	1.89×	2.96×	5.40×
512	0.32×	1.09×	2.35×	3.44×	5.87×
819	0.34×	1.06×	2.49×	3.94×	6.81×

Table 4: OpenMP speedup over the sequential baseline at query count 1, across query lengths and thread counts.

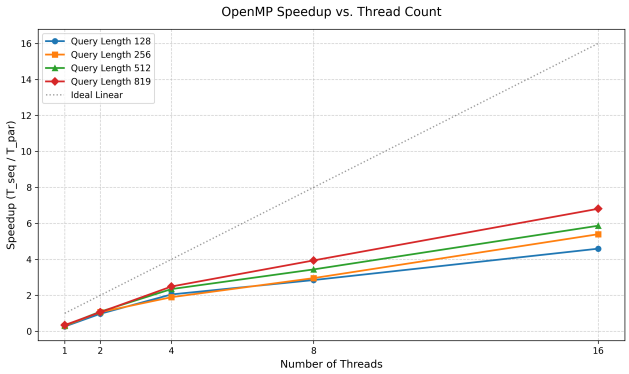


Figure 1: Speedup relative to thread count.

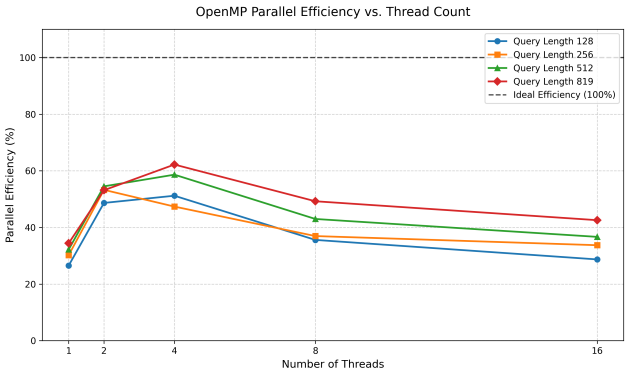


Figure 2: Efficiency of algorithm parallel implementation.

4.2.2 Scaling behavior

Beyond one thread, performance improves substantially across all thread counts, including beyond the physical core count of 8. At $T = 16$, the 8 logical threads added via SMT continue to provide meaningful throughput gains.

As visible in Figure 1, at query length 819 the speedup at $T = 2$ is nearly negligible at $1.06\times$, suggesting that OpenMP's threading overhead and dynamic scheduling costs dominate at this granularity. Scaling to $T = 4$ yields $2.49\times$, and $T = 8$ reaches $3.94\times$, reflecting sub-linear growth consistent with Amdahl's law. The transition from $T = 8$ to $T = 16$ produces the largest jump in the speedup factor, reaching $6.81\times$ — nearly doubling the throughput of the previous configuration.

4.2.3 Effect of query length

Longer queries yield higher speedups at equal thread counts. At $T = 16$, the speedup over sequential is $4.59\times$ for query length 128 but $6.81\times$ for query length 819. This occurs because longer queries generate more arithmetic work per window, which better amortizes the per-thread scheduling and synchronization overhead. In other words, the ratio of useful computation to parallel overhead increases with query length.

4.2.4 Parallel efficiency

Figure 2 shows parallel efficiency, defined as $E = S_P/P$ where S is the speedup and P is the thread count. At $T = 8$, efficiency ranges between 36% and 49% depending on query length. This sub-ideal efficiency reflects several concurrent factors: the fixed overhead of forking and joining the parallel region five times per function call, dynamic scheduling overhead, and residual load imbalance introduced by early abandonment — windows with a rapidly accumulating SAD terminate early, leaving some threads idle while others complete full evaluations. The `schedule(dynamic, 256)` clause mitigates the load imbalance but cannot eliminate it entirely, as the granularity of rebalancing is bounded by the chunk size.

4.3. CUDA Version

4.3.1 Absolute performance

The CUDA implementation achieves dramatically lower execution times across all configurations, ranging from 3.2 ms (query length 128, 1 query) to 34.1 ms (query length 819, 4 queries). These figures contrast with sequential times of 77.3 ms and 1972.0 ms respectively, underlining the advantage of massively parallel GPU execution.

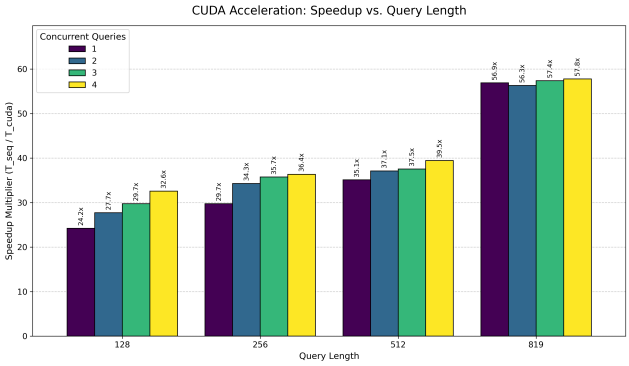


Figure 3: CUDA performance speedup multiplier versus the sequential baseline, evaluated across varying query lengths and concurrent query counts.

4.3.2 Speedup scaling with query length

A notable characteristic of the CUDA implementation is that its speedup over the sequential baseline *increases* with query length, as shown in Figure 3. With one query, the speedup is $24.2\times$ for length 128, rising to $35.1\times$ for length 512 and $56.9\times$ for length 819.

This trend might be explained by arithmetic intensity. For each candidate window, every thread performs `QUERY_LENGTH` iterations of the SAD accumulation loop against data already resident in `__shared__` memory, while the query itself is read from `__constant__` memory and broadcast once per warp. As query length grows, the number of floating-point operations per thread increases proportionally, whereas the cost of cooperatively loading the shared memory window and issuing the constant memory broadcast remains fixed. This improves the ratio of compute to memory traffic, allowing the GPU's execution units to remain better utilized and increasing the speedup over the sequential baseline.

4.3.3 Scaling with query count

Unlike the OpenMP version, whose speedup varies inconsistently with query count — showing no clear trend across thread counts and query lengths, and occasionally degrading at high query counts for long sequences — the CUDA implementation maintains a near-constant speedup across query counts. Going from 1 to 4 queries at length 819, GPU wall time grows from 8.6 ms to 34.1 ms — a factor of $3.98\times$ for a $4\times$ increase in work, reflecting linear scaling. This contrasts with the sequential baseline, which also scales linearly by construction (the function is invoked once per query), meaning the speedup ratio remains stable at approximately $57\times$. The advantage over OpenMP lies not in simultaneous query execution — the GPU's SM capacity is already saturated by the data-dimension blocks alone — but in the absence of per-query scheduling overhead and load imbalance: all query blocks are submitted in a single kernel launch and interleaved by the hardware scheduler at no additional cost to the host.

4.4. Comparisons

Table 5 summarizes the wall-clock speedup of the best OpenMP configuration ($T = 16$) and CUDA relative to the sequential baseline across all tested parameter combinations.

Length	Count	OMP-16 ($\times$)	CUDA ($\times$)
128	1	4.59	24.22
256	1	5.40	29.74
512	1	5.87	35.10
819	1	6.81	56.92
128	2	4.70	27.73
256	2	5.37	34.28
512	2	5.51	37.10
819	2	6.23	56.32
128	3	4.73	29.74
256	3	5.16	35.73
512	3	4.81	37.54
819	3	6.49	57.38
128	4	5.02	32.57
256	4	5.35	36.36
512	4	6.71	39.46
819	4	4.95	57.77

Table 5: Relative speedups of the OpenMP (16 threads) and CUDA (Block Size 512) implementations compared to the sequential baseline.

The most significant finding is that the performance gap between OpenMP and CUDA widens as query length increases. At query length 128 with one query, CUDA outperforms 16-thread OpenMP by a factor of $5.3\times$ (3.2 ms vs 16.8 ms) as it can be seen in Figure 4. At query length 819 with four queries, this advantage grows to $11.7\times$ (34.1 ms vs 398.6 ms). This divergence is caused by different bottlenecks: OpenMP is limited by the number of physical cores, whereas CUDA becomes more efficient at longer query lengths due to improved memory hierarchy utilization.

It is also worth noting that OpenMP at $T = 16$ outperforms the sequential baseline across all tested configurations, making it a practical choice when a GPU is unavailable. However, the speedup varies across parameter combinations — ranging from $4.59\times$ to $6.81\times$ — reflecting sensitivity to load distribution and thread overhead. For the maximum tested configuration (length 819, 4 queries), CUDA

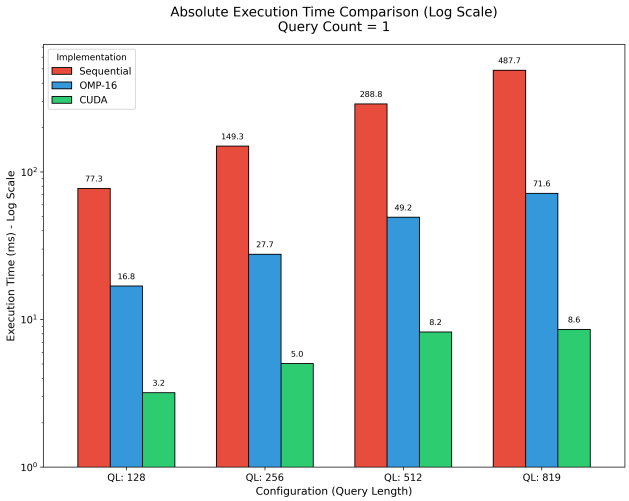


Figure 4: Absolute execution time comparison between all implementations using fixed query count.

completes in 34.1 ms compared to OpenMP's 398.6 ms — an order-of-magnitude difference that highlights the suitability of GPU acceleration for large-scale pattern recognition workloads.

5. Conclusion

In this study, we explored the acceleration of a pattern recognition algorithm across 5-channel time series data using the Sum of Absolute Differences metric. By establishing a correct sequential baseline, we successfully extended the algorithm to leverage both multi-core CPU (OpenMP) and many-core GPU (CUDA) architectures.

Our analysis highlights that while OpenMP provides a straightforward path to parallelization with considerable speedups, early abandonment algorithms require careful dynamic scheduling to prevent load imbalance. Conversely, CUDA delivers maximum throughput and the lowest execution times, provided the memory hierarchy (constant and shared memory) is properly utilized to feed the computational units.